

Principles of Distributed Database Systems

M. Tamer Özsu
Patrick Valduriez

Outline

- Introduction
- Distributed and Parallel Database Design
- Distributed Data Control
- Distributed Query Processing
- Distributed Transaction Processing
- Data Replication
- Database Integration – Multidatabase Systems
- **Parallel Database Systems**
- Peer-to-Peer Data Management
- Big Data Processing
- NoSQL, NewSQL and Polystores
- Web Data Management

Outline

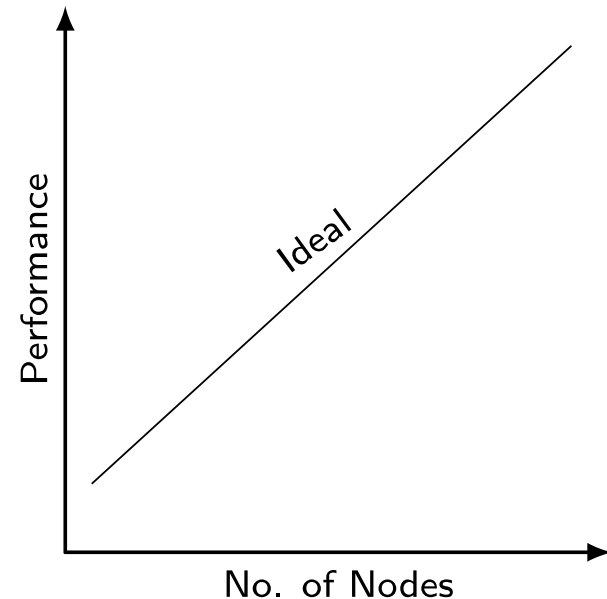
- Parallel Database Systems
 - ▣ Parallel Architectures
 - ▣ Data Placement
 - ▣ Query Processing
 - ▣ Load Balancing and Fault-tolerance
 - ▣ Database Clusters

Objectives of Parallel Systems

- High-performance using parallelism
 - High throughput for OLTP loads
 - Lots of short transactions, which update a few data
 - Low response time for OLAP queries
 - Large queries, which read lots of data
- High availability and reliability through data replication and failover
- Extensibility and scalability by adding resources
 - Processors, memory, disk, network

Extensibility

- Ideal: linear speed-up
 - Linear increase of performance by growing the components
 - For a fixed database size and load



Speed-up Limits

■ Hardware/software

- As we add more resources, arbitration conflicts increase
 - E.g. Access to the bus by processors

■ Application

- Only part of a program can be parallelized
- Recall: Amdahl's law that gives the maximum speed-up
 - Seq = fraction of code that cannot be parallelized

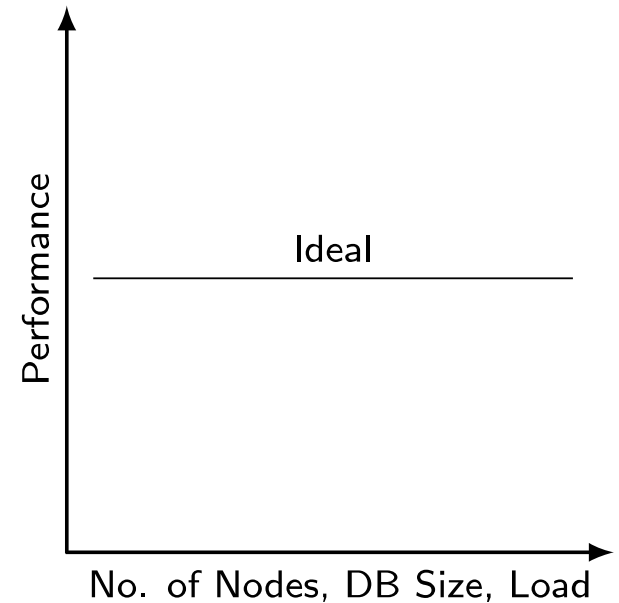
$$\frac{1}{Seq + \frac{1 - Seq}{NbProc}}$$

Examples

- $Seq=0, NbProc=4 \Rightarrow \text{speed-up} = 4$
- $Seq=30\%, NbProc=4 \Rightarrow \text{speed-up} = 2,1$
- $Seq=30\%, NbProc=8 \Rightarrow \text{speed-up} = 2,5$

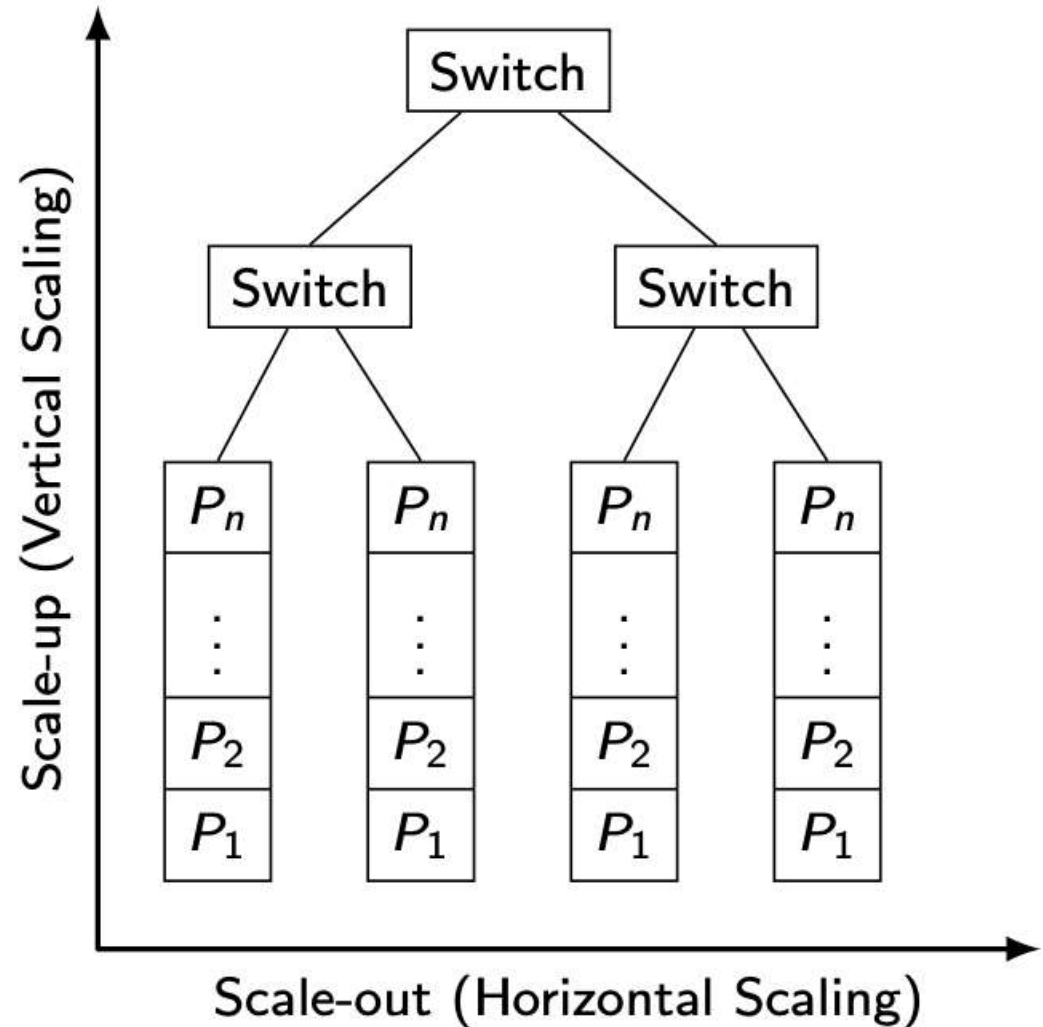
Scalability

- Ideal: linear scale-up
 - Sustained performance for a linear increase of database size and load
 - By proportional increase of components



Vertical vs Horizontal Scaleup

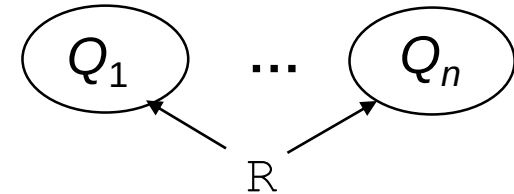
- Typically in a computer cluster



Data-based Parallelism

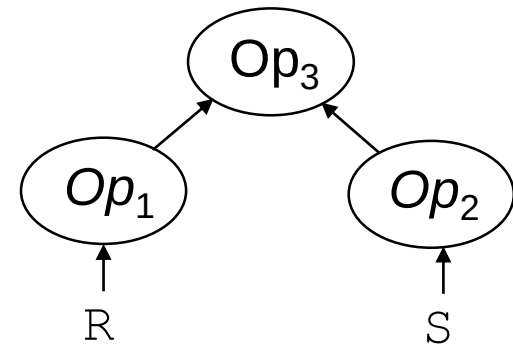
■ Inter-query

- ❑ Different queries on the same data
- ❑ For concurrent queries



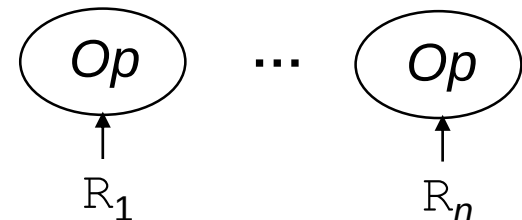
■ Inter-operation

- ❑ Different operations of the same query on different data
- ❑ For complex queries



■ Intra-operation

- ❑ The same operation on different data
- ❑ For large queries



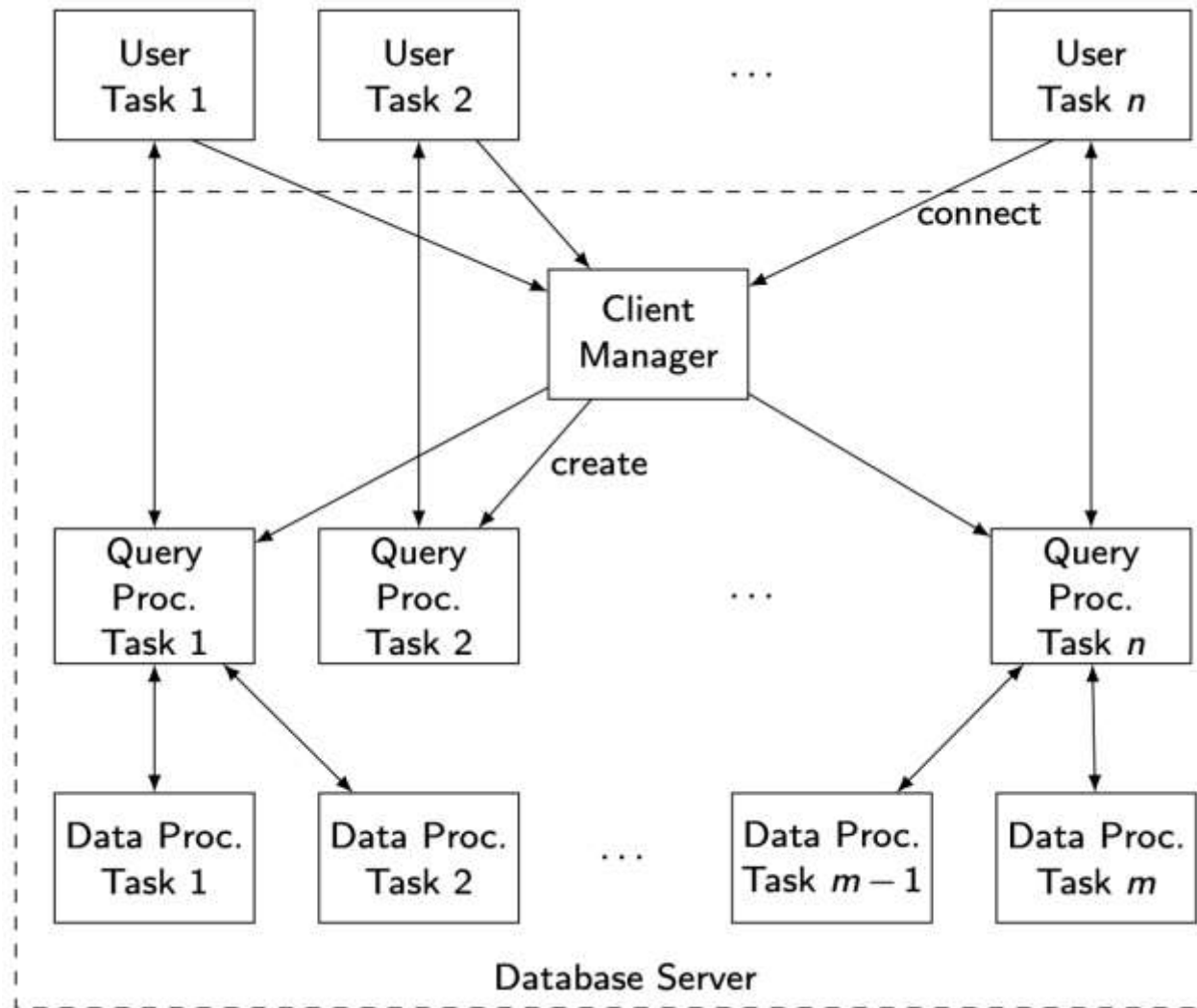
Barriers to Parallelism

- **Startup**
 - The time needed to start a parallel operation may dominate the actual computation time
- **Interference**
 - When accessing shared resources, each new process slows down the others (hot spot problem)
- **Skew**
 - The response time of a set of parallel processes is the time of the slowest one
- **Parallel data management techniques intend to overcome these barriers**

Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

Parallel DBMS – general architecture



Parallel DBMS Functions

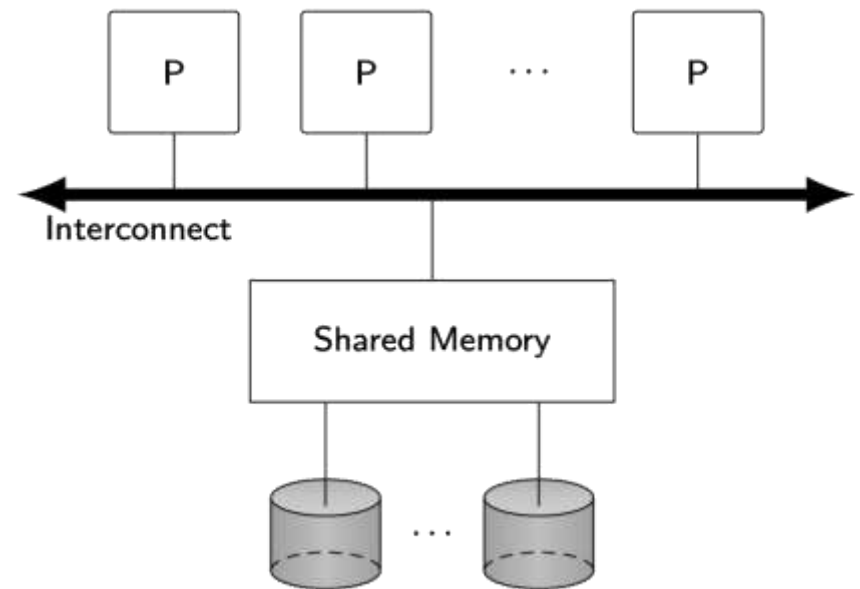
- Client manager
 - ❑ Support for client interactions and user sessions
 - ❑ Load balancing
- Query processor
 - ❑ Compilation and optimization
 - ❑ Catalog and metadata management
 - ❑ Semantic data control
 - ❑ Execution control
- Data processor
 - ❑ Execution of DB operations
 - ❑ Transaction management
 - ❑ Data management

Parallel System Architectures

- Shared memory (SM)
 - Uniform Memory Architecture (UMA)
 - Non-Uniform Memory Architecture (NUMA)
- Shared disk (SD)
- Shared nothing (SN)

UMA

- Physical memory shared by all processors
 - ❑ Symmetric multiprocessor (SMP) or multicore processor
 - ❑ Constant access time
- Examples
 - ❑ XPRS, Volcano, DBS3
- Assessment
 - + Simplicity, load balancing, fast communication
 - Network cost, low extensibility



NUMA

- Shared-memory vs. distributed memory
 - Mixes two different aspects : addressing and memory
 - Addressing: single address space vs multiple address spaces
 - Physical memory: central vs distributed
- NUMA = single address space on distributed physical memory
 - Eases application portability
 - Extensibility
- The most successful NUMA is Cache Coherent NUMA (CC-NUMA)

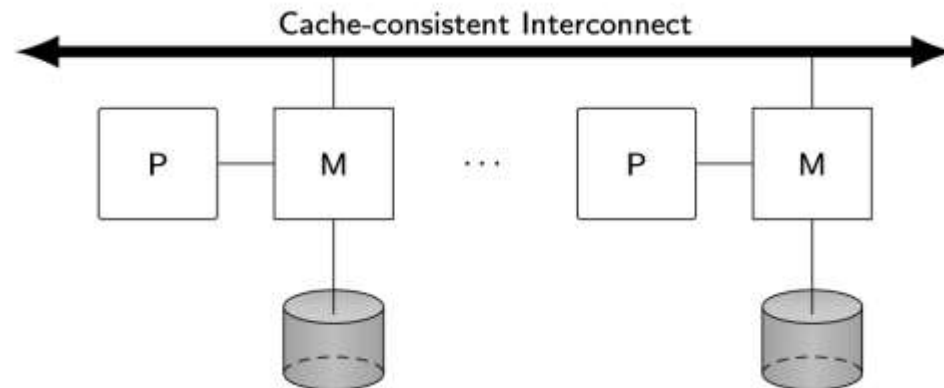
CC-NUMA

■ Principle

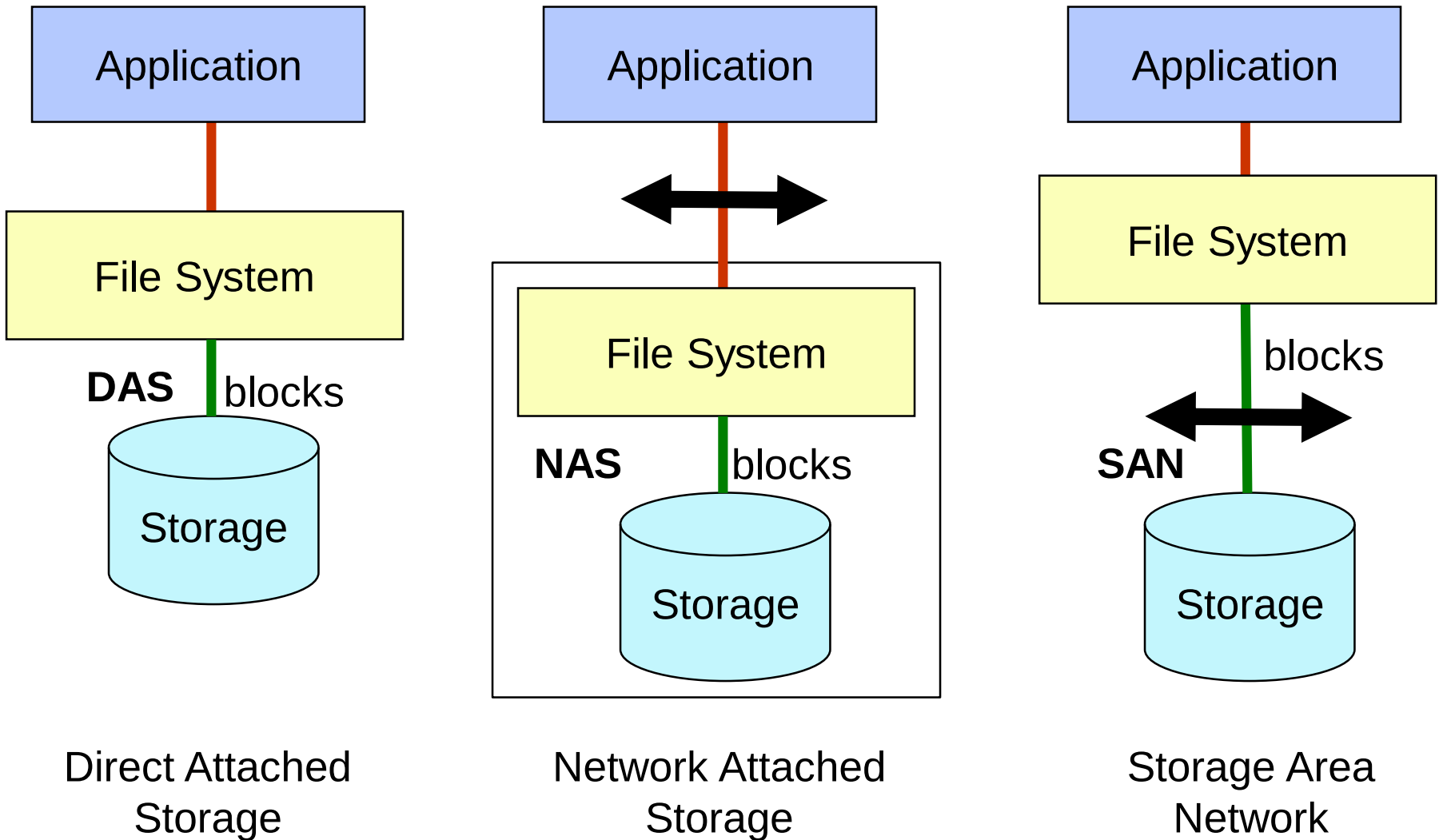
- ❑ Main memory physically distributed (as in shared-nothing)
- ❑ However, any processor has access to all other processors' memories

■ Cache consistency

- ❑ Special consistent cache interconnect
 - Remote memory access very efficient, only a few times (typically between 2 and 3 times) the cost of local access
- ❑ Exploit Remote Direct Memory Access (RDMA)
 - Now provided by low latency cluster interconnects such as Infiniband and Myrinet

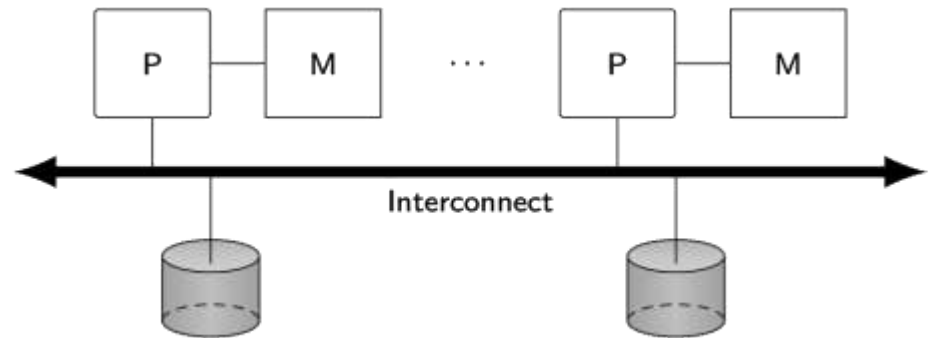


Storage: DAS vs NAS vs SAN



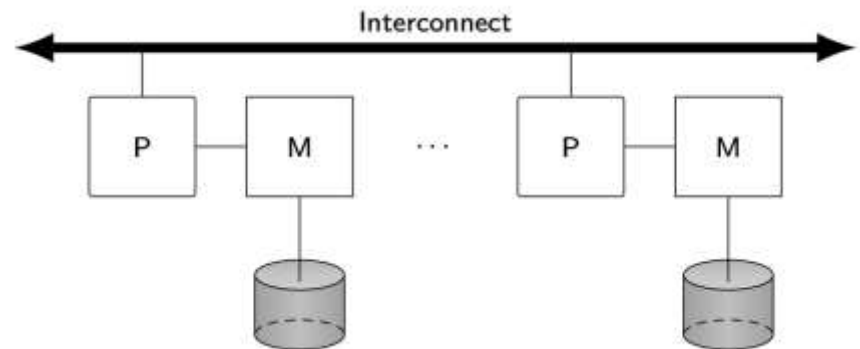
Shared-Disk

- Shared disk, private memory
 - ▣ SAN
 - ▣ Cache coherency
- Examples
 - ▣ Oracle RAC et Exadata
 - ▣ IBM PowerHA
- Assessment
 - + Simplicity for admin.
 - Network cost (SAN), scalability



Shared-Nothing

- No sharing of either disk or memory
 - ▣ Data partitioning
- Examples
 - ▣ DB2 DPF, SQL Server Parallel DW, Teradata, MySQLcluster
 - ▣ NoSQL, NewSQL
- Assessment
 - + Scalability, cost/performance
 - Complex (distributed updates)



Parallel DBMS Techniques

- Data placement
 - Data partitioning
 - Replication
- Parallel query processing
 - Parallel algorithms for relational operators
 - Query optimization
 - Load balancing
- Transaction management
 - Similar to distributed transaction management
 - But need to scale up to many nodes
 - Fault-tolerance and failover

Outline

■ Parallel Database Systems

- ❑ Parallel Architectures
- ❑ Data Placement
- ❑ Query Processing
- ❑ Load Balancing and Fault-tolerance
- ❑ Database Clusters

Data Partitioning

- Each relation is divided in n partitions (subrelations), where n is a function of relation size and access frequency
- Horizontal partitioning (see Chapter 2)
 - Round-robin
 - Maps i -th element to node $i \bmod n$
 - Simple but only exact-match queries
 - Hash
 - Only exact-match queries but small index
 - Range
 - Supports range queries but large index

Partitioning Functions

■ Hashing

- ❑ (k, v) assigned to node $h(k)$
- ❑ Exact match queries
- ❑ Problem with skew distribution

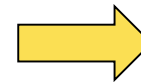


2				
2				
4				
4				
10				
12				

1				
11				
11				
13				

■ Range

- ❑ (k, v) to the node that holds k 's interval
- ❑ Exact match and range queries
- ❑ Needs an index on key



1				
2				
2				
4				
4				

10				
11				
11				
12				
13				

Replicated Data Partitioning

- High-availability requires data replication
 - Simple solution is mirrored disks
 - Hurts load balancing when one node fails
 - More elaborate solutions achieve load balancing
 - Interleaved partitioning (Teradata)
 - Chained partitioning (Gamma)

Interleaved Partitioning

Node	1	2	3	4
Primary copy	R_1	R_2	R_3	R_4
Backup copies	$R_{2,1}$ $R_{3,1}$ $R_{4,1}$	$R_{1,1}$ $R_{3,2}$ $R_{4,2}$	$R_{1,2}$ $R_{2,2}$ $R_{4,3}$	$R_{1,3}$ $R_{2,3}$ $R_{3,3}$

Chained Partitioning

Node	1	2	3	4
Primary copy	R_1	R_2	R_3	R_4
Backup copies	R_4 R_3	R_1 R_4	R_2 R_1	R_3 R_2